



ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO

FACULTAD DE INFORMÁTICA Y ELÉCTRICA

CARRERA DE SOFTWARE

Asignatura: Aplicaciones Informáticas II

Tema: Diseño arquitectónico de la aplicación

Fecha de entrega: 21 – 04 – 2026

Nombre:

Andrés Sebastián Peralta Ramos (7356)

Período académico:

Marzo 2026 – Julio 2026

1. Introducción

El presente documento establece el diseño arquitectónico de la aplicación web para la gestión de perfiles académicos mediante microcredenciales e insignias digitales en la Escuela Superior Politécnica de Chimborazo (ESPOCH). Se fundamenta en los lineamientos técnicos y requisitos especificados en la Especificación de Requisitos de Software (SRS), garantizando coherencia entre las necesidades institucionales y la solución tecnológica propuesta.

El propósito de este documento es definir la arquitectura de software del sistema, estableciendo los componentes principales, las tecnologías seleccionadas, los patrones de diseño adoptados y los mecanismos de comunicación entre capas.

Asimismo, este documento facilita la validación técnica por parte de los involucrados, y proporciona a los futuros mantenedores del sistema una visión clara de la estructura interna, las interfaces y las decisiones de diseño críticas. La audiencia principal incluye:

- **Equipo de desarrollo:** Responsables de la codificación, integración y pruebas técnicas.
- **Director de proyecto:** Para verificar el cumplimiento de los objetivos arquitectónicos y académicos.
- **Stakeholders institucionales:** Como respaldo técnico de la solución entregada.

2. Descripción general

La información aquí expuesta sirve como marco de referencia para comprender cómo la solución se integra con los procesos académicos de la ESPOCH y cómo responde a las necesidades de certificación y portabilidad de competencias.

2.1. Perspectiva del sistema

La aplicación web para la gestión de perfiles académicos mediante microcredenciales e insignias digitales es un producto de software independiente, desarrollado específicamente para cubrir las necesidades de certificación académica de la Escuela Superior Politécnica de Chimborazo.

Aunque opera como un sistema autónomo, está diseñado para integrarse conceptualmente al ecosistema tecnológico institucional, actuando como un subsistema especializado en la emisión, validación y portabilidad de credenciales académicas bajo estándares internacionales.

El sistema se estructura bajo una arquitectura cliente-servidor en tres capas claramente diferenciadas:

- **Capa de presentación (Frontend):** Interfaz web desarrollada en Angular, accesible mediante navegadores compatibles con estándares ECMAScript, que interactúa directamente con los actores del sistema (Emisor, Receptor y Administrador).
- **Capa de lógica de negocio (Backend):** Servidor de aplicaciones implementado en Node.js con Express, encargado de gestionar los procesos de autenticación, la validación de metadatos, la aplicación de firmas criptográficas y el cumplimiento del estándar Open Badges 3.0, en otras palabras, administra la lógica del negocio.
- **Capa de persistencia (Base de datos):** Sistema de gestión de bases de datos relacional PostgreSQL, responsable de almacenar de forma segura y estructurada la información de usuarios, microcredenciales, insignias emitidas y metadatos asociados.

Actualmente, el sistema no contempla una integración directa con las plataformas académicas legacy de la institución. La gestión de usuarios, la carga de datos y la validación de criterios se realizan de manera manual y controlada a través de los roles definidos, garantizando independencia operativa durante la fase inicial de despliegue.

2.2. Funcionalidades del sistema

El sistema proporciona una herramienta integral para la gestión del ciclo de vida de las microcredenciales e insignias digitales, desde su creación hasta su visualización y exportación por parte de los receptores. Las funcionalidades están organizadas en cuatro módulos principales:

- **Autenticación y gestión de usuarios:** Permite el acceso seguro al sistema mediante credenciales institucionales, validando la identidad y redirigiendo al panel correspondiente según el rol asignado (Emisor, Receptor o Administrador). Incluye mecanismos de control de acceso basados en roles (RBAC), gestión del ciclo de vida de cuentas (CRUD) por parte del Administrador, y configuración dinámica de permisos que se aplican inmediatamente sobre la sesión activa del usuario.
- **Administración de microcredenciales:** Facilita al Emisor el registro de nuevas microcredenciales, definiendo competencias, criterios de evaluación, metadatos académicos bajo el estándar Open Badges 3.0 y el diseño de la insignia digital asociada. Las microcredenciales quedan en estado “Pendiente de aprobación” hasta que el Administrador las valide. Una vez aprobadas, el sistema permite modificar su estado, visualizar listados con filtros de búsqueda, o aplicar eliminación lógica preservando el historial de insignias ya emitidas.
- **Emisión y validación de insignias:** Habilita al Emisor para otorgar insignias digitales a uno o varios Receptores que cumplan los criterios de evaluación. El sistema aplica automáticamente una firma criptográfica conforme a Open Badges 3.0, registra la emisión con fecha y hora, y genera una URL pública persistente para verificación externa. Además, permite consultar el historial de emisiones, filtrar resultados y ejecutar

procesos de revocación con justificación obligatoria, notificando al Receptor y actualizando el estado de la credencial en tiempo real.

- **Perfil académico y portafolio:** Ofrece al Receptor un espacio digital para visualizar, organizar y gestionar las insignias recibidas. El portafolio permite la organización por fecha o categorías personalizables, la búsqueda y filtrado de credenciales, la exportación en formatos estándar compatibles con repositorios externos, y la gestión de una hoja de vida académica editable. El sistema controla la visibilidad pública del perfil sin alterar los registros originales de emisión, garantizando tanto la personalización del usuario como la integridad de la trazabilidad institucional.

Estas funcionalidades operan de manera coordinada bajo la arquitectura en capas descrita, asegurando que cada proceso cumpla con los requisitos de adecuación funcional, seguridad y mantenimiento establecidos en la especificación de requisitos.

3. Selección de la arquitectura

3.1. Arquitectura seleccionada: Cliente-Servidor en 3 capas

Para el desarrollo de la aplicación web de gestión de perfiles académicos mediante microcredenciales e insignias digitales, se ha seleccionado una arquitectura cliente-servidor distribuida en tres capas claramente diferenciadas. Esta decisión responde directamente a los lineamientos técnicos establecidos en el SRS (SW-001, SW-002) y a la necesidad de garantizar una estructura modular, segura y mantenible.

La arquitectura se organiza de la siguiente manera:

- **Capa de presentación (Frontend):** Desarrollada en Angular con TypeScript, se ejecuta en el navegador del cliente y es responsable de la interacción con el usuario, la validación

inicial de formularios, la gestión de sesiones en el lado cliente y la renderización de las interfaces específicas por rol (RIU-001 a RIU-004).

- **Capa de lógica de negocio (Backend):** Implementada en Node.js con Express.js, actúa como intermediario entre el cliente y la base de datos. Gestiona la autenticación y autorización (JWT/RBAC), procesa las reglas de negocio, valida metadatos conforme al estándar Open Badges 3.0, aplica la firma criptográfica y expone una API REST segura bajo el protocolo HTTPS (COM-001, RNF-007).
- **Capa de persistencia (Base de datos):** Basada en PostgreSQL, almacena de forma estructurada y relacional la información de usuarios, microcredenciales, insignias emitidas, metadatos JSON-LD y registros operativos. Garantiza la integridad referencial y soporta consultas eficientes para la visualización del portafolio y la trazabilidad de emisiones (RNF-002, SW-002).

La comunicación entre capas se realiza mediante solicitudes HTTP/HTTPS estandarizadas, utilizando JSON como formato principal de intercambio de datos y JSON-LD para la estructuración semántica de las credenciales verificables (COM-002).

3.2. Justificación de la decisión

La selección de esta arquitectura se fundamenta en los siguientes criterios técnicos y metodológicos, alineados con los requisitos del proyecto:

- El documento de requisitos establece explícitamente el uso de Angular, Express.js y PostgreSQL (SW-001, SW-002). La arquitectura en 3 capas es el patrón natural que permite integrar estos componentes de forma desacoplada, respetando las responsabilidades de cada tecnología.

- La separación estricta entre presentación, lógica y datos permite que cada capa evolucione de manera independiente. Esto facilita la localización de errores, la actualización de dependencias y la incorporación de nuevas funcionalidades sin impactar el sistema completo.
- Centralizar la validación de credenciales, la aplicación de firmas criptográficas y la gestión de roles en el backend garantiza que la lógica sensible no se exponga en el cliente.
- La capa de backend actúa como orquestador para garantizar que cada insignia emitida cumpla estrictamente con Open Badges 3.0 (SW-003), validando estructuras JSON-LD antes de la firma y generando URLs públicas de verificación (RF-012).

3.3. Ventajas

Esta selección presenta beneficios claros y consideraciones que deben gestionarse durante el desarrollo:

- Cada capa tiene un propósito definido, lo que permite asignar tareas específicas a miembros del equipo según su especialidad (frontend, backend o base de datos).
- La API REST expuesta por el backend puede ser consumida por otros clientes en el futuro sin modificar la lógica de negocio.
- Facilita la implementación centralizada de requisitos de seguridad (HTTPS, hashing, RBAC) y trazabilidad, ya que el backend actúa como punto único de control.

Esta arquitectura garantiza que el sistema cumpla con los requisitos funcionales y no funcionales establecidos, manteniendo un equilibrio entre rigor técnico y viabilidad práctica.

4. Patrones y estilos arquitectónicos

Para garantizar la calidad, mantenibilidad y escalabilidad del sistema, se han seleccionado patrones y estilos arquitectónicos ampliamente validados en la ingeniería de software moderna. Su aplicación responde directamente a las restricciones tecnológicas (SW-001, SW-002) y al requisito de modularidad (RNF-008), asegurando una separación clara de responsabilidades y una comunicación estandarizada entre los componentes del sistema.

4.1. Arquitectura basada en componentes y servicios

En la capa de presentación se adopta la arquitectura nativa de Angular, fundamentada en componentes y servicios. Este enfoque permite una organización modular del código, facilitando la reutilización, el mantenimiento y la escalabilidad del frontend:

- **Componentes:** Unidades básicas de construcción de la interfaz de usuario, responsables de renderizar las vistas (RIU-001 a RIU-004), capturar las interacciones del usuario y coordinar la presentación de datos. Cada componente gestiona su propia vista mediante plantillas HTML y estilos encapsulados, siguiendo el principio de responsabilidad única.
- **Servicios:** Clases singleton que encapsulan la lógica de negocio, el acceso a datos y las operaciones transversales (autenticación, gestión de estado, comunicación con APIs). Los servicios se inyectan en los componentes mediante el sistema de inyección de dependencias de Angular, promoviendo la separación de responsabilidades y facilitando las pruebas unitarias.
- **Comunicación entre componentes:** Se implementa mediante bindings de entrada/salida (@Input/@Output), servicios compartidos con BehaviorSubject para estado reactivo, y routing con paso de parámetros para navegación entre vistas.

Esta arquitectura garantiza que la lógica de presentación esté desacoplada de la lógica de negocio, facilitando la mantenibilidad del código (RNF-008) y permitiendo que los componentes sean reutilizables en diferentes módulos del sistema.

4.2. API REST

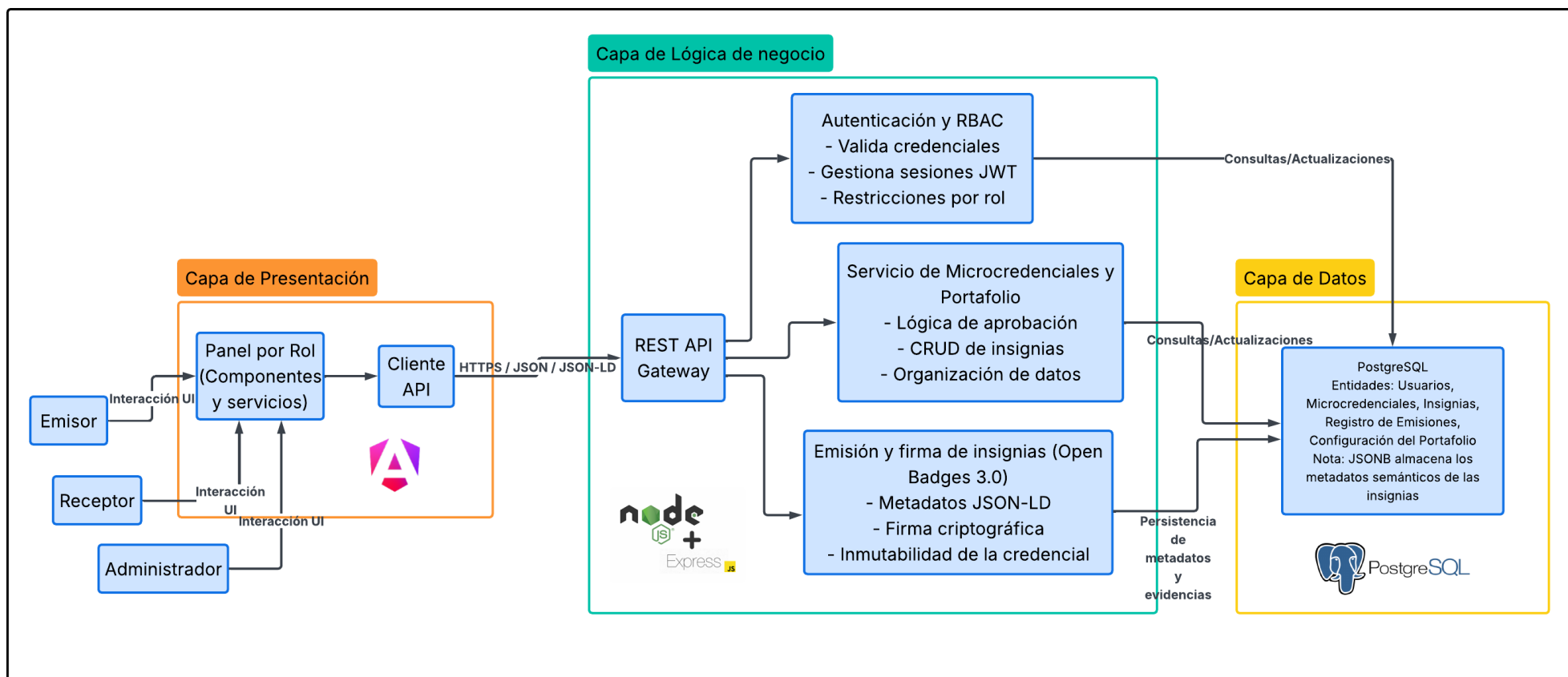
La comunicación entre el frontend (Angular) y el backend (Express) se estructura bajo el estilo REST (Representational State Transfer), cumpliendo con los principios de cliente-servidor, statelessness e interfaz uniforme. Los endpoints se diseñan como recursos identificados por URLs semánticas, utilizando verbos HTTP estándar (GET, POST, PUT, DELETE) para operaciones CRUD y flujos específicos.

Precisión sobre el formato de intercambio:

Conforme al requisito COM-002 y al estándar Open Badges 3.0, el uso de JSON se restringe exclusivamente a la estructura de datos de insignias digitales y microcredenciales, garantizando interoperabilidad semántica mediante JSON-LD. Para el resto de la comunicación API, se emplea JSON estándar como payload de intercambio, optimizando el rendimiento y simplificando el parsing en Angular y Express. Esta diferenciación evita sobrecarga técnica innecesaria mientras asegura el cumplimiento normativo para credenciales verificables (RF-012, RNF-006).

Todas las comunicaciones se protegen mediante HTTPS (COM-001), y las respuestas incluyen códigos de estado HTTP estandarizados, cabeceras de seguridad y mensajes de error controlados para no exponer trazas internas (RNF-007).

5. Diagrama de la arquitectura propuesta



El diagrama presentado ilustra la estructura lógica de la aplicación web, distribuida en tres capas desacopladas tal como se definió en la selección arquitectónica. A continuación, se describen los componentes y flujos principales representados:

Capa de Presentación

Desarrollada en Angular y TypeScript, actúa como el punto de entrada para los tres actores del sistema: Emisor, Receptor y Administrador. Esta capa orquesta la navegación mediante un componente de "Panel por Rol", que adapta dinámicamente la interfaz según los permisos del usuario obtenidos vía token JWT. Incluye módulos específicos para la gestión de credenciales, visualización del portafolio y un Cliente API que centraliza todas las peticiones salientes bajo el protocolo seguro HTTPS, garantizando la confidencialidad en tránsito (RNF-007).

Capa de Lógica de negocio

Implementada con Node.js y Express, recibe las solicitudes a través de un Gateway REST API que rutea y valida las peticiones. Internamente, la lógica se organiza en servicios especializados agrupados dentro del dominio de negocio:

- **Módulo de autenticación y RBAC:** Intermediario de seguridad que valida credenciales, gestiona sesiones y aplica restricciones de acceso basadas en roles antes de permitir el acceso a recursos sensibles.
- **Servicio de microcredenciales y portafolio:** Gestionan las operaciones CRUD, la lógica de aprobación institucional y la organización de datos académicos para la visualización y exportación.
- **Emisión y firma de insignias (Open Badges 3.0):** Componente crítico responsable de estructurar los metadatos conforme al estándar, aplicar la firma criptográfica y asegurar la integridad e inmutabilidad de la credencial antes de su persistencia.

Capa de datos (Data Layer)

Basada en PostgreSQL, almacena la información persistente del sistema. Las entidades principales incluyen Usuarios, Microcredenciales, Insignias, Registro de Emisiones y Configuración del Portafolio. La comunicación con el backend se optimiza mediante consultas relacionales eficientes y el uso de tipos de datos JSONB para almacenar los metadatos semánticos de las insignias, facilitando consultas flexibles sin perder la estructura relacional de la base de datos.

El flujo de comunicación se establece mediante HTTPS para toda la transferencia de datos, mientras que el intercambio de información de credenciales utiliza estrictamente el formato JSON-LD (COM-002), asegurando la interoperabilidad y verificabilidad exigida por el estándar internacional.

6. Conclusiones

La arquitectura definida para la aplicación web de gestión de perfiles académicos mediante microcredenciales e insignias digitales en la ESPOCH responde de manera integral a los requisitos funcionales y no funcionales establecidos en la Especificación de Requisitos de Software (SRS). A través de la adopción de un patrón cliente-servidor en tres capas, se logra una separación clara de responsabilidades que facilita el desarrollo paralelo, la mantenibilidad del código y la escalabilidad controlada del sistema.

La selección de tecnologías cumple con las restricciones tecnológicas del proyecto (SW-001, SW-002) y además aprovecha ecosistemas maduros y ampliamente documentados, reduciendo riesgos de implementación y facilitando la incorporación de nuevos desarrolladores. La comunicación estandarizada mediante API REST y el uso de JSON/JSON-LD exclusivamente

para la estructura de credenciales (COM-002) garantizan interoperabilidad con el estándar Open Badges 3.0 sin sobrecargar innecesariamente el resto de la comunicación del sistema.

La arquitectura en capas permite centralizar la lógica de seguridad en el backend, cumpliendo con los requisitos de autenticación segura (RNF-004), control de acceso basado en roles (RNF-005) y protección de datos en tránsito (RNF-007).